

TP Calidad y Automatización - CI/CD

Objetivo: Implementar un pipeline de calidad sobre la aplicación desarrollada en el TP2, asegurando que cada cambio en el código sea validado y desplegado automáticamente.

Equipos: Dos personas, se sugiere mantener los del TP2.

Entrega: Repositorio de GitHub (con documentación requerida) + defensa oral.

Duración: 4 semanas.

Marco Teórico

Hasta ahora construyeron una aplicación funcional y la desplegaron. El siguiente paso es asegurar su despliegue: que el código que sube a producción haya sido verificado automáticamente, y que si algo falla, lo sepan antes que el usuario.

Eso es todo un nuevo concepto e incluso un nuevo rol que generalmente en TIC no han observado nunca: DevOps. Es el que se encarga que el producto logrado por los desarrolladores pueda ser desplegado correctamente, eso no se garantiza sólo con tests, los test solo validan ciertos casos de uso específicos, depende también de cómo se organiza el trabajo: cómo se proponen cambios, cómo se revisan, cómo se comunican los problemas. En este TP el repositorio debe reflejar un flujo de trabajo robusto, profesional de principio a fin.

[Videito](#) sobre DevOps

Modalidad de trabajo

Todo el trabajo del TP debe estar trazado en GitHub, no solo en el código:

Issues: cada funcionalidad, mejora o bug a resolver debe tener su issue correspondiente antes de empezar a trabajar. El issue debe tener título descriptivo, una breve descripción y estar asignado a un integrante.

Pull Requests: ningún cambio se mergea directo a main ni a develop. Todo pasa por un PR que referencia el issue que resuelve (ej. closes #12). El otro integrante del equipo debe revisar y aprobar el PR antes de mergearlo.

Branch naming: las ramas deben seguir una convención consistente, por ejemplo feature/nombre-feature, fix/nombre-bug. Documentarla en el README.

Revisión de código: los PRs deben tener al menos un comentario de revisión real (no aprobaciones vacías). Se valorará que las revisiones sean concretas y útiles.

El historial de issues y PRs es parte de la entrega.

Requisitos mínimos (funcionales)

1. Pipeline CI/CD

Workflow de GitHub Actions que se dispare en cada push o PR a main

El workflow debe ejecutar: lint → tests → build → deploy a Vercel

El deploy a producción solo debe ocurrir si todos los pasos anteriores pasan

La URL de producción debe estar documentada en el README

2. Testing

Al menos 2 tests unitarios con Vitest (o Jest) sobre lógica de negocio o funciones utilitarias

Al menos 1 test E2E que cubra el flujo principal de la aplicación (ej. login → crear/ver una entidad)

Los tests deben correr dentro del pipeline de CI

3. Documentación de calidad — CALIDAD.md

El repositorio debe incluir un archivo CALIDAD.md en la raíz del proyecto. No es un archivo de configuración ni un log automático: es un documento escrito por el equipo que explica, con sus propias palabras, las decisiones tomadas para asegurar la calidad del producto.

Debe cubrir al menos:

Estrategia general: qué enfoque eligieron para garantizar calidad y por qué. No alcanza con listar herramientas; deben explicar el razonamiento detrás de cada elección.

Herramientas seleccionadas: qué usan para tests unitarios, E2E, lint, CI/CD, y por qué eligieron esas y no otras. Si evaluaron alternativas y las descartaron, mencionarlo.

Tests desarrollados: listado de los tests implementados con una descripción de qué caso de uso cubre cada uno y qué comportamiento valida. Ejemplo: "Test E2E: usuario no autenticado es redirigido al login al intentar acceder a /dashboard".

Casos de uso críticos: cuáles son los flujos de la aplicación que consideran más importantes proteger con tests y por qué los priorizaron sobre otros.

Pipeline de CI/CD: descripción de los pasos del workflow, qué hace cada uno y qué decisión de diseño tomaron (ej. por qué el deploy solo corre si los tests pasan, qué pasa si falla el lint).

Limitaciones y deuda técnica: qué quedó sin cubrir, qué mejorarían con más tiempo, qué sabían que era frágil y aceptaron como riesgo consciente.

Opcional extra:

- Integración de Sentry u otro servicio de error monitoring con al menos un error capturado y documentado en el CALIDAD.md
 - Cobertura de tests superior al 60% en las funciones de negocio (reporte generado con vitest --coverage), con el resultado comentado en el CALIDAD.md
 - Uso de un agente de IA (Codex, Cursor, etc.) para generación de tests, con sección en el CALIDAD.md explicando qué generó, qué modificaron y por qué
 - Uso de GitHub Projects o el tablero de issues para organizar el trabajo del equipo durante el TP (kanban básico: To Do / In Progress / Done)
 - Implementación de PR templates con checklist de revisión
-

Anexo:

Conceptos a evaluar

- El pipeline de CI corre y falla correctamente si un test falla (se puede demostrar con un PR de prueba)
- Los tests tienen sentido: prueban comportamiento real, no son triviales
- El deploy es automático y la aplicación en producción está actualizada
- El flujo de trabajo en GitHub es visible y consistente: issues trackeados, PRs con revisión, historial limpio
- El CALIDAD.md refleja comprensión real del sistema: decisiones justificadas, casos de uso pensados, limitaciones honestas
- En la defensa oral: cada integrante puede explicar qué hace el pipeline, qué testea cada test, recorrer el historial de PRs e issues, y defender las decisiones documentadas en el [CALIDAD.md](#)

Nota sobre herramientas de IA

Pueden usar Copilot, Codex, Cursor o cualquier agente para ayudarse a escribir tests. Lo que se evalúa es que entiendan lo que entregan: en la defensa se les puede pedir que expliquen cualquier línea de cualquier test. Lo mismo aplica si usaron IA para redactar partes del CALIDAD.md: el documento puede estar bien escrito, pero las decisiones que describe tienen que poder defenderlas en persona.